

11.5 PLANNING WITH PROPOSITIONAL LOGIC

We saw in Chapter 10 that planning can be done by proving a theorem in situation calculus. That theorem says that, given the initial state and the successor-state axioms that describe the effects of actions, the goal will be true in a situation that results from a certain action sequence. As early as 1969, this approach was thought to be too inefficient for finding interesting plans. Recent developments in efficient reasoning algorithms for propositional logic (see Chapter 7) have generated renewed interest in planning as logical reasoning.

The approach we take in this section is based on testing the **satisfiability** of a logical sentence rather than on proving a theorem. We will be finding models of propositional sentences that look like this:

$$\text{initial state} \wedge \text{all possible action descriptions} \wedge \text{goal} .$$

The sentence will contain proposition symbols corresponding to every possible action occurrence; a model that satisfies the sentence will assign *true* to the actions that are part of a correct plan and *false* to the others. An assignment that corresponds to an incorrect plan will not be a model, because it will be inconsistent with the assertion that the goal is true. If the planning problem is unsolvable, then the sentence will be unsatisfiable.

Describing planning problems in propositional logic

The process we will follow to translate STRIPS problems into propositional logic is a textbook example (so to speak) of the knowledge representation cycle: We will begin with what seems to be a reasonable set of axioms, we will find that these axioms allow for spurious unintended models, and we will write more axioms.

Let us begin with a very simple air transport problem. In the initial state (time 0), plane P_1 is at *SFO* and plane P_2 is at *JFK*. The goal is to have P_1 at *JFK* and P_2 at *SFO*; that is, the planes are to change places. First, we will need distinct proposition symbols for assertions about each time step. We will use superscripts to denote the time step, as in Chapter 7. Thus, the initial state will be written as

$$At(P_1, SFO)^0 \wedge At(P_2, JFK)^0 .$$

(Remember that $At(P_1, SFO)^0$ is an atomic symbol.) Because propositional logic has no closed-world assumption, we must also specify the propositions that are *not* true in the initial

state. If some propositions are unknown in the initial state, then they can be left unspecified (the **open world assumption**). In this example we specify:

$$\neg At(P_1, JFK)^0 \wedge \neg At(P_2, SFO)^0 .$$

The goal itself must be associated with a particular time step. Since we do not know *a priori* how many steps it takes to achieve the goal, we can try asserting that the goal is true in the initial state, time $T = 0$. That is, we assert $At(P_1, JFK)^0 \wedge At(P_2, SFO)^0$. If that fails, we try again with $T = 1$, and so on until we reach the minimum feasible plan length. For each value of T , the knowledge base will include only sentences covering the time steps from 0 up to T . To ensure termination, an arbitrary upper limit, $T_{\max}$, is imposed. This algorithm is shown in Figure 11.15. An alternative approach that avoids multiple solution attempts is discussed in Exercise 11.17.

```

function SATPLAN(problem,  $T_{\max}$ ) returns solution or failure
  inputs: problem, a planning problem
            $T_{\max}$ , an upper limit for plan length

  for  $T = 0$  to  $T_{\max}$  do
    cnf, mapping  $\leftarrow$  TRANSLATE-TO-SAT(problem,  $T$ )
    assignment  $\leftarrow$  SAT-SOLVER(cnf)
    if assignment is not null then
      return EXTRACT-SOLUTION(assignment, mapping)
  return failure

```

Figure 11.15 The SATPLAN algorithm. The planning problem is translated into a CNF sentence in which the goal is asserted to hold at a fixed time step T and axioms are included for each time step up to T . (Details of the translation are given in the text.) If the satisfiability algorithm finds a model, then a plan is extracted by looking at those proposition symbols that refer to actions and are assigned *true* in the model. If no model exists, then the process is repeated with the goal moved one step later.

The next issue is how to encode action descriptions in propositional logic. The most straightforward approach is to have one proposition symbol for each action occurrence; for example, $Fly(P_1, SFO, JFK)^0$ is true if plane P_1 flies from SFO to JFK at time 0. As in Chapter 7, we write propositional versions of the successor-state axioms developed for the situation calculus in Chapter 10. For example, we have

$$At(P_1, JFK)^1 \Leftrightarrow (At(P_1, JFK)^0 \wedge \neg(Fly(P_1, JFK, SFO)^0 \wedge At(P_1, JFK)^0)) \vee (Fly(P_1, SFO, JFK)^0 \wedge At(P_1, SFO)^0) . \quad (11.1)$$

That is, plane P_1 will be at JFK at time 1 if it was at JFK at time 0 and didn't fly away, or it was at SFO at time 0 and flew to JFK . We need one such axiom for each plane, airport, and time step. Moreover, each additional airport adds another way to travel to or from a given airport and hence adds more disjuncts to the right-hand side of each axiom.

With these axioms in place, we can run the satisfiability algorithm to find a plan. There ought to be a plan that achieves the goal at time $T = 1$, namely, the plan in which the two

planes swap places. Now, suppose the KB is

$$\text{initial state} \wedge \text{successor-state axioms} \wedge \text{goal}^1, \quad (11.2)$$

which asserts that the goal is true at time $T = 1$. You can check that the assignment in which

$$\text{Fly}(P_1, \text{SFO}, \text{JFK})^0 \text{ and } \text{Fly}(P_2, \text{JFK}, \text{SFO})^0$$

are true and all other action symbols are false is a model of the KB. So far, so good. Are there other possible models that the satisfiability algorithm might return? Indeed, yes. Are all these other models satisfactory plans? Alas, no. Consider the rather silly plan specified by the action symbols

$$\text{Fly}(P_1, \text{SFO}, \text{JFK})^0 \text{ and } \text{Fly}(P_1, \text{JFK}, \text{SFO})^0 \text{ and } \text{Fly}(P_2, \text{JFK}, \text{SFO})^0.$$

This plan is silly because plane P_1 starts at SFO , so the action $\text{Fly}(P_1, \text{JFK}, \text{SFO})^0$ is infeasible. Nonetheless, the plan is a model of the sentence in Equation (11.2)! That is, it is consistent with everything we have said so far about the problem. To understand why, we need to look more carefully at what the successor-state axioms (such as Equation (11.1)) say about actions whose preconditions are not satisfied. The axioms *do* predict correctly that nothing will happen when such an action is executed (see Exercise 11.15), but they *do not* say that the action cannot be executed! To avoid generating plans with illegal actions, we must add **precondition axioms** stating that an action occurrence requires the preconditions to be satisfied.⁶ For example, we need

$$\text{Fly}(P_1, \text{JFK}, \text{SFO})^0 \Rightarrow \text{At}(P_1, \text{JFK})^0.$$

Because $\text{At}(P_1, \text{JFK})^0$ is stated to be false in the initial state, this axiom ensures that every model also has $\text{Fly}(P_1, \text{JFK}, \text{SFO})^0$ set to false. With the addition of precondition axioms, there is exactly one model that satisfies all of the axioms when the goal is to be achieved at time 1, namely the model in which plane P_1 flies to JFK and plane P_2 flies to SFO . Notice that this solution has two parallel actions, just as with GRAPHPLAN or POP.

More surprises emerge when we add a third airport, LAX . Now, each plane has two actions that are legal in each state. When we run the satisfiability algorithm, we find that a model with $\text{Fly}(P_1, \text{SFO}, \text{JFK})^0$ and $\text{Fly}(P_2, \text{JFK}, \text{SFO})^0$ and $\text{Fly}(P_2, \text{JFK}, \text{LAX})^0$ satisfies all the axioms. That is, the successor-state and precondition axioms allow a plane to fly to two destinations at once! The preconditions for the two flights by P_2 are satisfied in the initial state; the successor-state axioms say that P_2 will be at SFO and LAX at time 1; so the goal is satisfied. Clearly, we must add more axioms to eliminate these spurious solutions. One approach is to add **action exclusion axioms** that prevent simultaneous actions. For example, we can insist on complete exclusion by adding all possible axioms of the form

$$\neg(\text{Fly}(P_2, \text{JFK}, \text{SFO})^0 \wedge \text{Fly}(P_2, \text{JFK}, \text{LAX})^0).$$

These axioms ensure that no two actions can occur at the same time. They eliminate all spurious plans, but also force every plan to be totally ordered. This loses the flexibility of partially ordered plans; also, by increasing the number of time steps in the plan, computation time may be lengthened.

⁶ Notice that the addition of precondition axioms means that we need not include preconditions for actions in the successor-state axioms.

Instead of complete exclusion, we can require only partial exclusion—that is, rule out simultaneous actions only if they interfere with each other. The conditions are the same as those for mutex actions: two actions cannot occur simultaneously if one negates a precondition or effect of the other. For example, $Fly(P_2, JFK, SFO)^0$ and $Fly(P_2, JFK, LAX)^0$ cannot both occur, because each negates the precondition of the other; on the other hand, $Fly(P_1, SFO, JFK)^0$ and $Fly(P_2, JFK, SFO)^0$ can occur together because the two planes do not interfere. Partial exclusion eliminates spurious plans without forcing a total ordering.

Exclusion axioms sometimes seem a rather blunt instrument. Instead of saying that a plane cannot fly to two airports at the same time, we might simply insist that no object can be in two places at once:

$$\forall p, x, y, t \quad x \neq y \Rightarrow \neg(At(p, x)^t \wedge At(p, y)^t) .$$

STATE CONSTRAINTS

This fact, combined with the successor-state axioms, *implies* that a plane cannot fly to two airports at the same time. Facts such as this are called **state constraints**. In propositional logic, of course, we have to write out all the ground instances of each state constraint. For the airport problem, the state constraint suffices to rule out all spurious plans. State constraints are often much more compact than action exclusion axioms, but they are not always easy to derive from the original STRIPS description of a problem.

To summarize, planning as satisfiability involves finding models for a sentence containing the initial state, the goal, the successor-state axioms, the precondition axioms, and either the action exclusion axioms or the state constraints. It can be shown that this collection of axioms is sufficient, in the sense that there are no longer any spurious “solutions.” Any model satisfying the propositional sentence will be a valid plan for the original problem—that is, every linearization of the plan is a legal sequence of actions that reaches the goal.

Complexity of propositional encodings

The principal drawback of the propositional approach is the sheer size of the propositional knowledge base that is generated from the original planning problem. For example, the action schema $Fly(p, a_1, a_2)$ becomes $T \times |Planes| \times |Airports|^2$ different proposition symbols. In general, the total number of action symbols is bounded by $T \times |Act| \times |O|^P$, where $|Act|$ is the number of action schemata, $|O|$ is the number of objects in the domain, and P is the maximum arity (number of arguments) of any action schema. The number of clauses is larger still. For example, with 10 time steps, 12 planes, and 30 airports, the complete action exclusion axiom has 583 million clauses.

Because the number of action symbols is exponential in the arity of the action schema, one answer might be to try to reduce the arity. We can do this by borrowing an idea from semantic networks (Chapter 10). Semantic networks use only binary predicates; predicates with more arguments are reduced to a set of binary predicates that describe each argument separately. Applying this idea to an action symbol such as $Fly(P_1, SFO, JFK)^0$, we obtain three new symbols:

$$\begin{aligned} Fly_1(P_1)^0 &: \text{plane } P_1 \text{ flew at time } 0 \\ Fly_2(SFO)^0 &: \text{the origin of the flight was } SFO \\ Fly_3(JFK)^0 &: \text{the destination of the flight was } JFK . \end{aligned}$$

SYMBOL SPLITTING

This process, called **symbol splitting**, eliminates the need for an exponential number of symbols. Now we only need $T \times |Act| \times P \times |O|$.

Symbol splitting by itself can reduce the number of symbols, but does not automatically reduce the number of axioms in the KB. That is, if each action symbol in each clause were simply replaced by a conjunction of three symbols, then the total size of the KB would remain roughly the same. Symbol splitting actually does reduce the size of the KB because some of the split symbols will be irrelevant to certain axioms and can be omitted. For example, consider the successor-state axiom in Equation (11.1), modified to include *LAX* and to omit action preconditions (which will be covered by separate precondition axioms):

$$At(P_1, JFK)^1 \Leftrightarrow (At(P_1, JFK)^0 \wedge \neg Fly(P_1, JFK, SFO)^0 \wedge \neg Fly(P_1, JFK, LAX)^0) \vee Fly(P_1, SFO, JFK)^0 \vee Fly(P_1, LAX, JFK)^0.$$

The first condition says that P_1 will be at *JFK* if it was there at time 0 and didn't fly from *JFK* to any other city, no matter which one; the second says it will be there if it flew to *JFK* from another city, no matter which one. Using the split symbols, we can simply omit the argument whose value does not matter:

$$At(P_1, JFK)^1 \Leftrightarrow (At(P_1, JFK)^0 \wedge \neg (Fly_1(P_1)^0 \wedge Fly_2(JFK)^0)) \vee (Fly_1(P_1)^0 \wedge Fly_3(JFK)^0).$$

Notice that *SFO* and *LAX* are no longer mentioned in the axiom. More generally, the split action symbols now allow the size of each successor-state axiom to be independent of the number of airports. Similar reductions occur with the precondition axioms and action exclusion axioms (see Exercise 11.16). For the case described earlier with 10 time steps, 12 planes, and 30 airports, the complete action exclusion axiom is reduced from 583 million clauses to 9,360 clauses.

There is one drawback: the split-symbol representation does not allow for parallel actions. Consider the two parallel actions $Fly(P_1, SFO, JFK)^0$ and $Fly(P_2, JFK, SFO)^0$. Converting to the split representation, we have

$$Fly_1(P_1)^0 \wedge Fly_2(SFO)^0 \wedge Fly_3(JFK)^0 \wedge Fly_1(P_2)^0 \wedge Fly_2(JFK)^0 \wedge Fly_3(SFO)^0.$$

It is no longer possible to determine what happened! We know that P_1 and P_2 flew, but we cannot identify the origin and destination of each flight. This means that a complete action exclusion axiom must be used, with the drawbacks noted previously.

Planners based on satisfiability can handle large planning problems—for example, finding optimal 30-step solutions to blocks-world planning problems with dozens of blocks. The size of the propositional encoding and the cost of solution are highly problem-dependent, but in most cases the memory required to store the propositional axioms is the bottleneck. One interesting finding from this work has been that backtracking algorithms such as DPLL are often better at solving planning problems than local search algorithms such as WALKSAT. This is because the majority of the propositional axioms are Horn clauses, which are handled efficiently by the unit propagation technique. This observation has led to the development of hybrid algorithms combining some random search with backtracking and unit propagation.